

Unit 2 notes

What is Database Design?

Database Design is a collection of processes that facilitate the designing, development, implementation and maintenance of enterprise data management systems. Properly designed database are easy to maintain, improves data consistency and are cost effective in terms of disk storage space. The database designer decides how the data elements correlate and what data must be stored. The main objectives of database designing are to produce logical and physical designs models of the proposed database system.

The logical model concentrates on the data requirements and the data to be stored independent of physical considerations. It does not concern itself with how the data will be stored or where it will be stored physically.

The physical data design model involves translating the logical design of the database onto physical media using hardware resources and software systems such as database management systems (DBMS).

What is good database design?

Certain principles guide the database design process. The first principle is that duplicate information (also called redundant data) is bad, because it wastes space and increases the likelihood of errors and inconsistencies. The second principle is that the correctness and completeness of information is important. If your database contains incorrect information, any reports that pull information from the database will also contain incorrect information. As a result, any decisions you make that are based on those reports will then be misinformed.

A good database design is, therefore, one that:

- Divides your information into subject-based tables to reduce redundant data.
- Provides Access with the information it requires to join the information in the tables together as needed.
- Helps support and ensure the accuracy and integrity of your information.
- Accommodates your data processing and reporting needs.

Significance of database design:

1. Consistency –Consistency refers to uniformity. A database design helps data to be stored which is relevant and required. It is one important feature in database design
2. Cascading –Consider a parent and a child record. The data present in child record is allowed if and only if the data is present in parent record. This prevent data repetition.
3. Redundancy –It is duplication or repetition of data. It makes the database more complex and takes up the storage unnecessarily. The solution to data redundancy is normalization. It also reduces the size of database.
4. Queries –A database design is said to be good, if its queries are simple and execution is faster.
5. Performance –The database design is credited if the database design is good enough.
6. Maintenance –The most important feature of a good database design. If it is a well designed database, it can be maintained well otherwise it might lead to many problems.

The design process

The design process consists of the following steps:

Determine the purpose of your database :

This helps prepare you for the remaining steps.

Find and organize the information required

Gather all of the types of information you might want to record in the database, such as product name and order number.

Divide the information into tables

Divide your information items into major entities or subjects, such as Products or Orders. Each subject then becomes a table.

Turn information items into columns

Decide what information you want to store in each table. Each item becomes a field, and is displayed as a column in the table. For example, an Employees table might include fields such as Last Name and Hire Date.

Specify primary keys

Choose each table's primary key. The primary key is a column that is used to uniquely identify each row. An example might be Product ID or Order ID.

Set up the table relationships

Look at each table and decide how the data in one table is related to the data in other tables. Add fields to tables or create new tables to clarify the relationships, as necessary.

Refine your design

Analyse your design for errors. Create the tables and add a few records of sample data. See if you can get the results you want from your tables. Make adjustments to the design, as needed.

Apply the normalization rules

Apply the data normalization rules to see if your tables are structured correctly. Make adjustments to the tables, as needed.

How to Design a SQL Database

The first step to designing any database in SQL is to identify what to include and what not to include. The next steps involve deciding how the included items relate to each other and then setting up tables accordingly.

To design a database in SQL, follow these basic steps:

1. Decide what objects you want to include in your database.
2. Determine which of these objects should be tables and which should be columns within those tables.
3. Define tables based on how you need to organize the objects.

Step 1: Define objects

The first step in designing a database is deciding which aspects of the system are important enough to include in the model. Treat each aspect as an object and create a list of all the objects you can think of. At this stage, don't try to decide how these objects relate to each other. Just try to list them all.

When you have a reasonably complete set of objects, move on to the next step: deciding how these objects relate to each other. Some of the objects are major entities that are crucial to giving you the results you want. Other objects are subsidiary to those major entities. Ultimately you may decide that some objects don't belong in the model at all.

Step 2: Identify tables and columns

Major entities translate into database tables. Each major entity has a set of attributes — the table columns. Many business databases, for example, have a CUSTOMER table that keeps track of customers' names, addresses, and other permanent information. Each attribute of a customer — such as name, street, city, state, zip code, phone number, and e-mail address — becomes a column (and a column heading) in the CUSTOMER table.

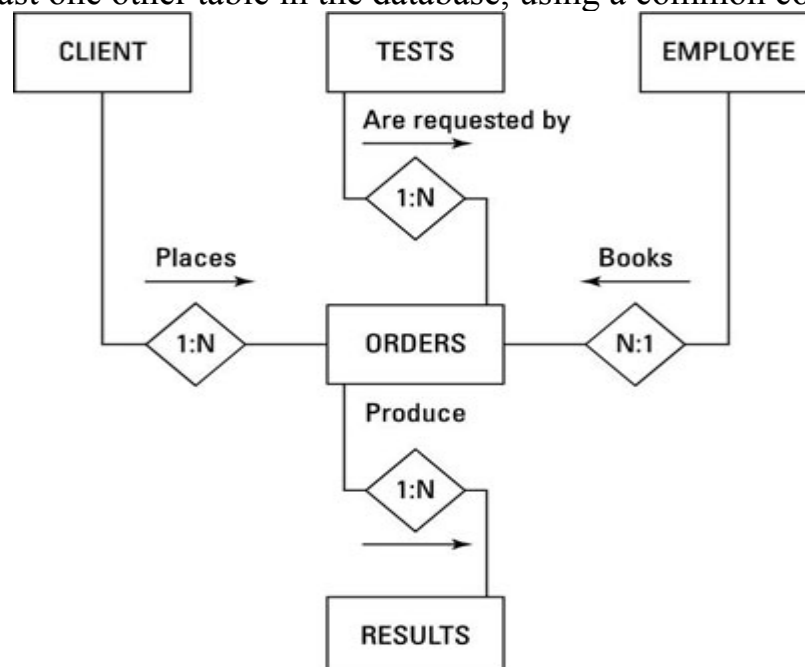
If you're hoping to find a set of rules to help you identify which objects should be tables and which of the attributes in the system belong to which tables, think again: You may have some reasons for assigning a particular attribute to one table and other reasons for assigning the same attribute to another table. You must base your judgment on two goals:

- The information you want to get from the database
- How you want to use that information

Step 3: Define tables

Now you want to define a table for each entity and a column for each attribute.

If you want a table to serve as an integral part of a relational database, link that table to at least one other table in the database, using a common column.



The links illustrate four different one-to-many relationships. The diamond in the middle of each relationship shows the maximum cardinality of each end of the relationship. The number 1 denotes the “one” side of the relationship, and N denotes the “many” side.

- One client can make many orders, but each order is made by one, and only one, client.
- Each test can appear on many orders, but each order calls for one, and only one, test.
- Each order is taken by one, and only one, employee (or salesperson), but each salesperson can take multiple orders.
- Each order can produce several preliminary test results and a final result, but each result is associated with one, and only one, order.

The attribute that links one table to another can have a different name in each table. Both attributes must, however, have matching data types.

DBMS Methodology (Conceptual):

The methodology is depicted as a bit by bit guide to the three main phases of database design, namely: conceptual, logical, and physical design.

The primary aim of each phase is as follows:

1. **Conceptual database design** - to build the conceptual representation of the database, which has the identification of the important entities, relationships, and attributes.
2. **Logical database design** - to convert the conceptual representation to the logical structure of the database, which includes designing the relations.
3. **Physical database design** - to decide how the logical structure is to be physically implemented (as base relations) in the target Database Management System (DBMS).

Introduction to the Database Design Methodology

A structured approach that uses procedures, techniques, tools, and documentation help to support and make possible the process of design is called Design Methodology.

A design methodology encapsulates various phases, each containing some stages, which guide the designer in the techniques suitable at each stage of the project. A design methodology also helps the designer to plan, manage, control, and evaluate database development and managing projects. Furthermore, it is a planned approach for analyzing and modeling a group of requirements for a database in a standardized and ordered manner.

Conceptual Database Design

In this design methodology, the process of constructing a model of the data is used in an enterprise, independent of all physical considerations. The conceptual database design phase starts with the formation of a conceptual data model of the enterprise that is entirely independent of implementation details such as the target DBMS, use of application programs, programming languages used, hardware platform, performance issues, or any other physical deliberations.

Critical Success Factors in Database Design

The following planning strategies are often critical to the success of database design:

- Deal with task interactively with the users as much as possible.
- Follow a prearranged methodology throughout the data modeling process.
- Make use of a data-driven approach.
- Incorporate structural and integrity considerations into the data models.
- Combine conceptualization, normalization, and transaction validation methods into the data modelling methodology.
- Use figures for representing as much of the data models as possible.
- Use a Database Design Language (DBDL) to represent additional data semantics that cannot usually be represented in a diagram.
- Build a data dictionary to add-on the data model diagrams and the DBDL.
- Be willing to repeat steps.

These factors are constructed into the methodology that is presented for database design.

What are the steps for Conceptual Database Design?

Conceptual database design steps are:

- Build a conceptual data model.
- Recognize entity types.
- Recognize the relationship types.
- Identify and connect attributes with entity or relationship types.
- Determine attribute domains.
- Determine candidate, primary, and alternate key attributes.
- Consider the use of improved modeling concepts (optional step)
- Check model for redundancy.
- Validate the conceptual model against user transactions.
- Review the conceptual data model with user.

Building a Conceptual Data Model:

The first step in conceptual database design is to build one (or more) conceptual data replica of the data requirements of the enterprise. A conceptual data model comprises these following elements:

- entity types

- types of relationship
- attributes and the various attribute domains
- primary keys and alternate keys
- integrity constraints

The conceptual data model is maintained by documentation, including ER diagrams and a data dictionary, which is produced throughout the development of the model.

Entity Relationship (ER) Model:

One of the most challenging phases of database design is the fact that designers, programmers, and end-users tend to view data and its use in various forms. Unfortunately, unless all the database learners gain a common understanding that reflects how the enterprise operates but the design you may produce will fail to meet the users' requirements. To ensure that you get a precise understanding of the nature of the data and how the enterprise uses it, you need to have a universal model for interaction that is non-technical and free of ambiguities and easily readable to both technical as well as non-technical members. So the ER (Entity Relationship) Model was designed and developed and is represented by an ER diagram.

ER modeling is a top-down structure to database design that begins with identifying the important data called entities and relationships in combination with the data that must be characterized in the model. Then database model designers can add more details such as the information they want to hold about the entities and relationships, which are the attributes and any constraints on the entities, relationships, and attributes. ER modeling is an important technique for any database designer to master and forms the basis of the methodology

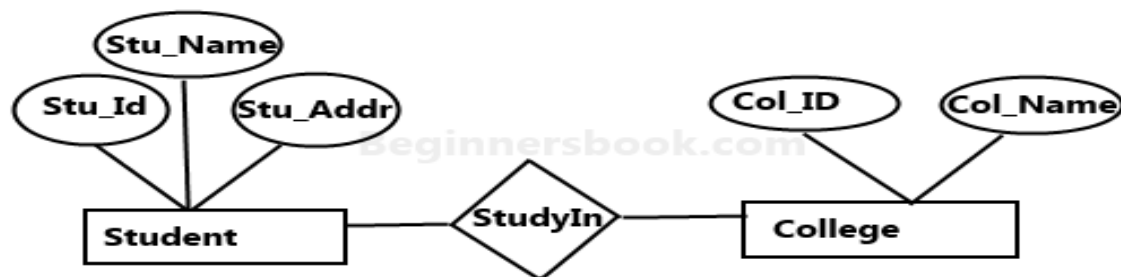
Entity Relationship Diagram (ER-Diagram)

ER-Diagram is a pictorial representation of data that describes how data is communicated and related to each other. Any object, such as entities, attributes of an entity, sets of relationship, and other attributes of relationship, can be characterized with the help of the ER diagram.

A simple ER Diagram:

In the following diagram we have two entities Student and College and their relationship. The relationship between Student and College is many to one as a

college can have many students however a student cannot study in multiple colleges at the same time. Student entity has attributes such as Stu_Id, Stu_Name & Stu_Addr and College entity has attributes such as Col_ID & Col_Name.



Sample E-R Diagram

Rectangle: Represents Entity sets.

Ellipses: Attributes

Diamonds: Relationship Set

Lines: They link attributes to Entity Sets and Entity sets to Relationship Set

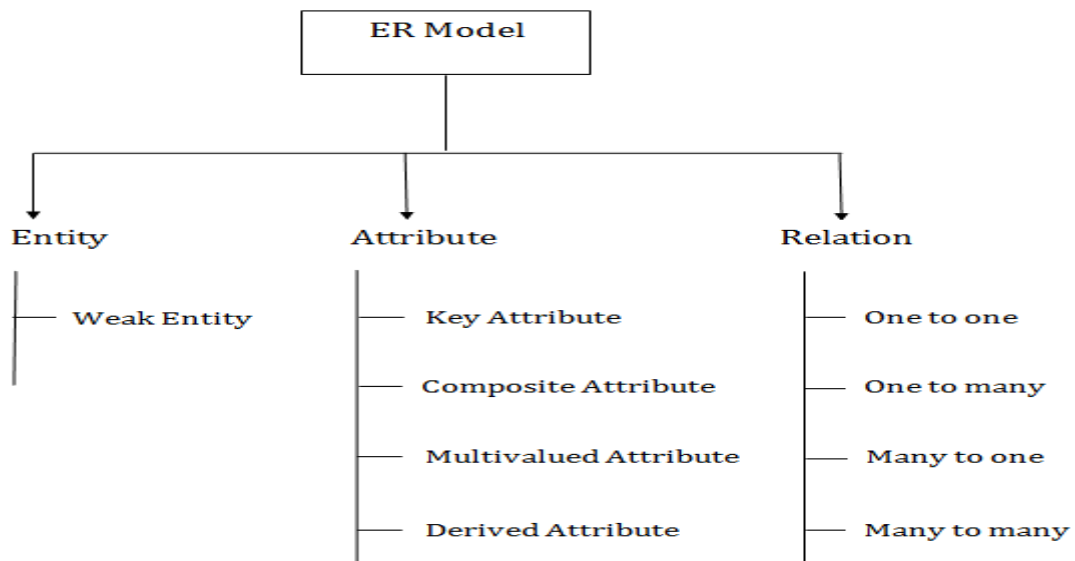
Double Ellipses: Multivalued Attributes

Dashed Ellipses: Derived Attributes

Double Rectangles: Weak Entity Sets

Double Lines: Total participation of an entity in a relationship set.

Components of a ER Diagram:



As shown in the above diagram, an ER diagram has three main components:

1. Entity
2. Attribute
3. Relationship

1. Entity

An entity is an object or component of data. An entity is represented as rectangle.

An Entity may be an object with a physical existence – a particular person, car, house, or employee – or it may be an object with a conceptual existence.

For example: In the following ER diagram we have two entities Student and College and these two entities have many to one relationship as many students study in a single college. We will read more about relationships later, for now focus on entities.

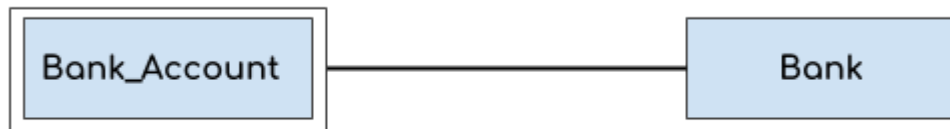


Beginnersbook.com

Weak Entity:

An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is

represented by a double rectangle. For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



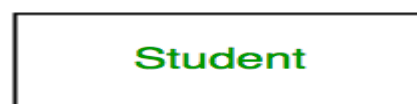
Beginnerbook.com

Entity type and entity set:

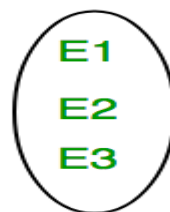
An entity type defines the collection of entities that have the same attributes.

The collection of all entities of a particular entity type in the database at any particular point in time is called an entity set.

An Entity is an object of Entity Type and set of all entities is called as entity set. e.g.; E1 is an entity having Entity Type Student and set of all students is called Entity Set. In ER diagram, Entity Type is represented as:



Entity Type



Entity Set

2. Attribute:

An attribute describes the property of an entity. For example, Roll_No, Name, DOB, Age, Address, Mobile_No are the attributes which defines entity type Student. An attribute is represented as Oval in an ER diagram. There are four types of attributes:

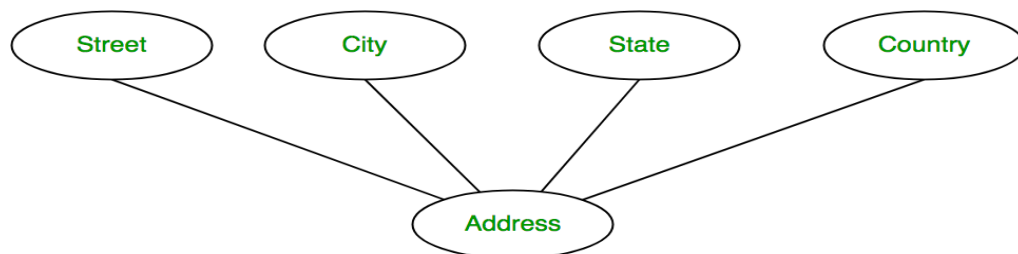
1. Key attribute
2. Composite attribute
3. Multivalued attribute

4. Derived attribute

Key attribute: The attribute which uniquely identifies each entity in the entity set is called key attribute. For example, Roll_No will be unique for each student. In ER diagram, key attribute is represented by an oval with underlying lines.



Composite attribute: An attribute composed of many other attribute is called as composite attribute. For example, Address attribute of student Entity type consists of Street, City, State, and Country. In ER diagram, composite attribute is represented by an oval comprising of ovals.



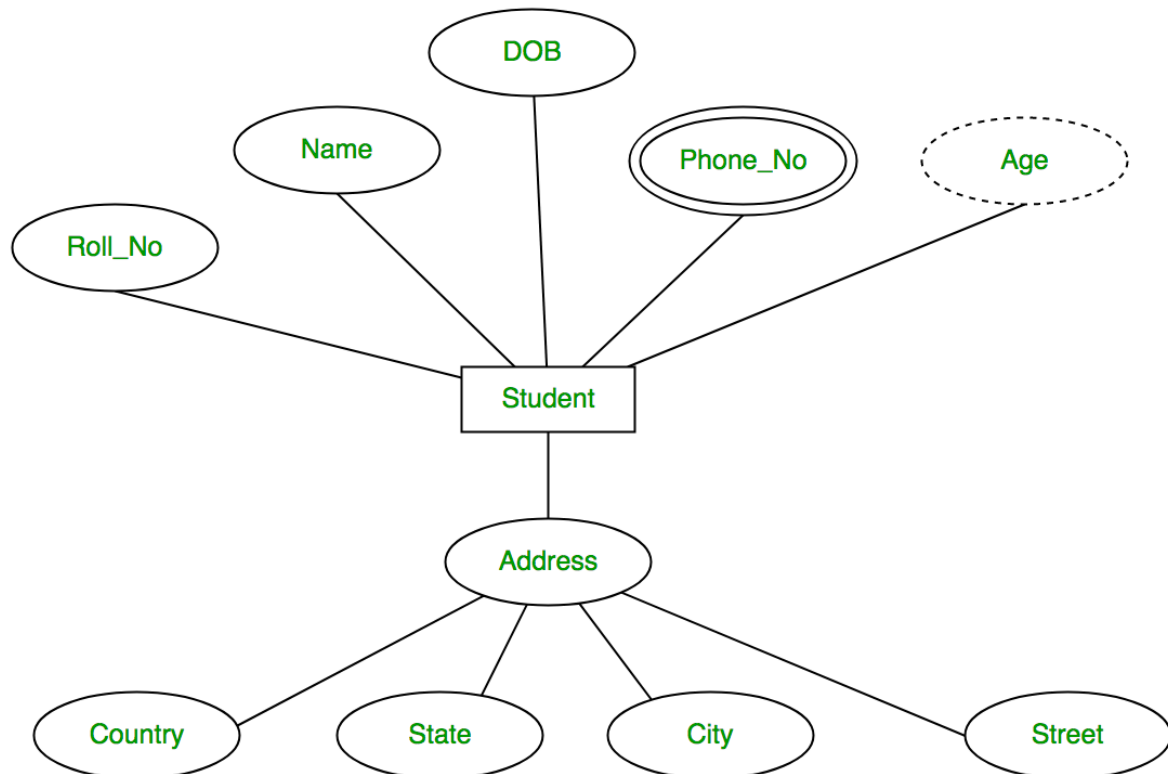
Multivalued attribute: An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, multivalued attribute is represented by double oval.



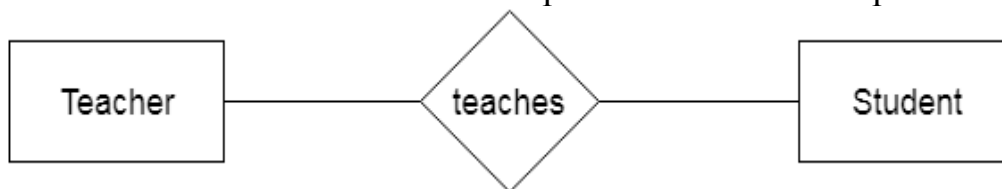
Derived attribute: An attribute which can be derived from other attributes of the entity type is known as derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, derived attribute is represented by dashed oval.



The complete entity type Student with its attributes can be represented as:



3.Relationship: A relationship is used to describe the relation between entities. Diamond or rhombus is used to represent the relationship.



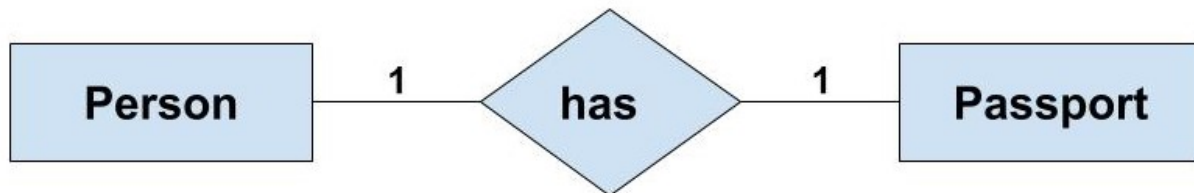
We have two entity types of 'Customer'(Customer_id, Name, City, Phone) and 'Account'(Account_no, Type, Balance). We store the data of 'Customer' in one table and his accounts details in the 'Account' table. Now, to link these two tables we need to insert the primary key 'Customer_id' of the 'Customer' table in the 'Account' table. This key acts as a foreign key for the 'Account' table and refers to a column with the same name in the 'Customer' table. This is how a relationship between two tables is established. There are three types of relationships that can exist between two entities.

- One-to-One Relationship
- One-to-Many or Many-to-One Relationship
- Many-to-Many Relationship

One-to-One Relationship:

Such a relationship exists when each record of one table is related to only one record of the other table.

For example, If there are two entities 'Person' (Id, Name, Age, Address) and 'Passport'(Passport_id, Passport_no). So, each person can have only one passport and each passport belongs to only one person.

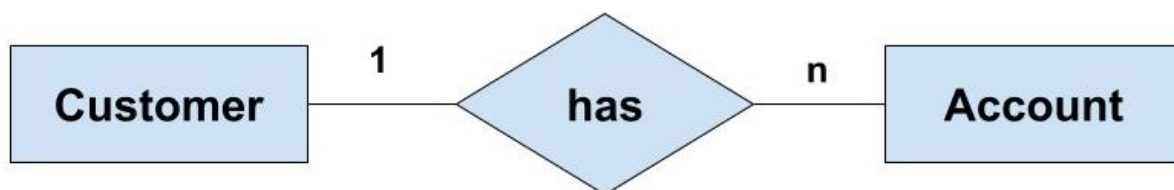


Such a relationship is not very common. However, such a relationship is used for security purposes. In the above example, we can easily store the passport id in the 'Person' table only. But, we make another table for the 'Passport' because Passport number may be sensitive data and it should be hidden from certain users. So, by making a separate table we provide extra security that only certain database users can see it.

One-to-Many or Many-to-One Relationship:

Such a relationship exists when each record of one table can be related to one or more than one record of the other table. This relationship is the most common relationship found. A one-to-many relationship can also be said as a many-to-one relationship depending upon the way we view it.

For example, If there are two entity type 'Customer' and 'Account' then each 'Customer' can have more than one 'Account' but each 'Account' is held by only one 'Customer'. In this example, we can say that each Customer is associated with many Account. So, it is a one-to-many relationship. But, if we see it the other way i.e many Account is associated with one Customer then we can say that it is a many-to-one relationship.

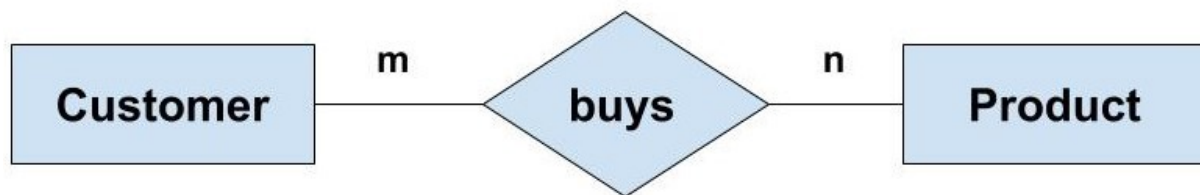


Many-to-Many Relationship :

Such a relationship exists when each record of the first table can be related to one or more than one record of the second table and a single record of the

second table can be related to one or more than one record of the first table. A many-to-many relationship can be seen as a two one-to-many relationship which is linked by a 'linking table' or 'associate table'. The linking table links two tables by having fields which are the primary key of the other two tables. We can understand this with the following example.

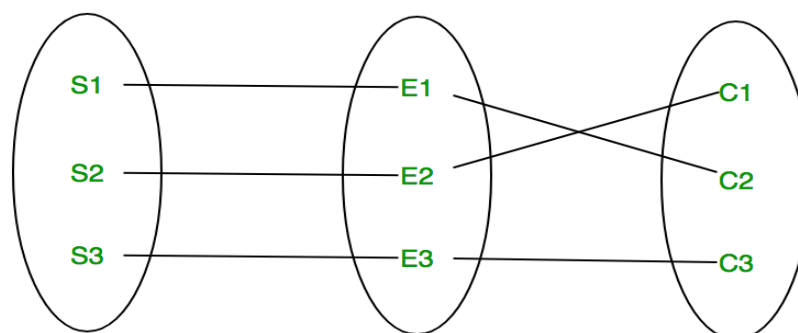
Example: If there are two entity type 'Customer' and 'Product' then each customer can buy more than one product and a product can be bought by many different customers.



Relationship type and relationship set: A relationship type represents the association between entity types. For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course. In ER diagram, relationship type is represented by a diamond and connecting the entities with lines.

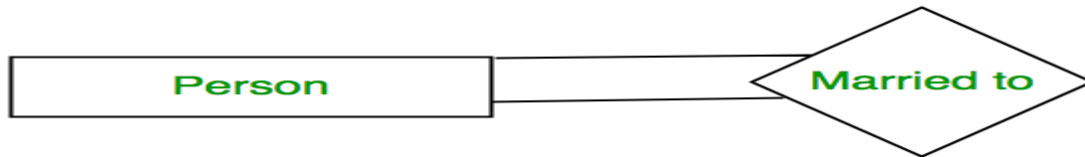


A set of relationships of same type is known as relationship set. The following relationship set depicts S1 is enrolled in C2, S2 is enrolled in C1 and S3 is enrolled in C3.



Degree of a relationship set: The number of different entity sets participating in a relationship set is called as degree of a relationship set.

1)Unary Relationship –When there is only ONE entity set participating in a relation, the relationship is called as unary relationship. For example, one person is married to only one person.



2) Binary Relationship –When there are TWO entities set participating in a relation, the relationship is called as binary relationship. For example, Student is enrolled in Course.



Role name: The role name signifies the role that a participating entity from the entity type plays in each relationship instance and helps to explain what that relationship means. For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of employee or worker and DEPARTMENT plays the role of department or employer.

Recursive relationships: Role names are not technically necessary in relationship types where all the participating entity types are distinct, since each participating entity type name can be used as the role name. However, in some cases the same entity type participates more than once in a relationship type in different roles. In such cases the role name becomes essential for distinguishing the meaning of the role that each participating entity plays. Such relationship types are called recursive relationships.

Example: The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity set. Hence, the EMPLOYEE entity type participates twice in SUPERVISION: once in the role of supervisor (or boss), and once in the role of supervisee (or subordinate).

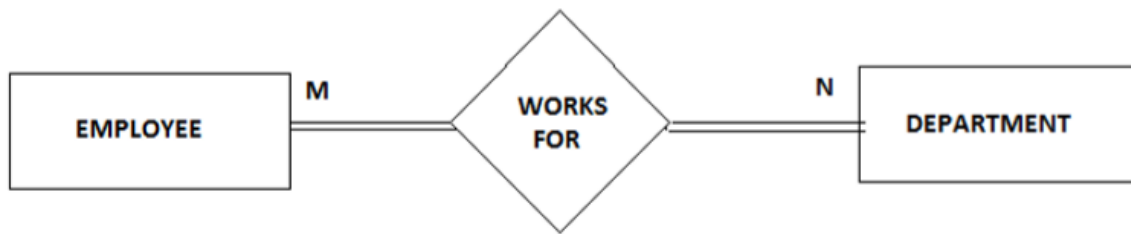
Participation Constraint:

In a Relationship, Participation constraint specifies the existence of an entity when it is related to another entity in a relationship type. It is also called minimum cardinality constraint.

This constraint specifies the number of instances of an entity that can participate in a relationship type.

There are two types of Participation constraint –

Total Participation – Each entity in the entity set is involved in at least one relationship in a relationship set i.e. the number of relationship in every entity is involved is greater than 0.

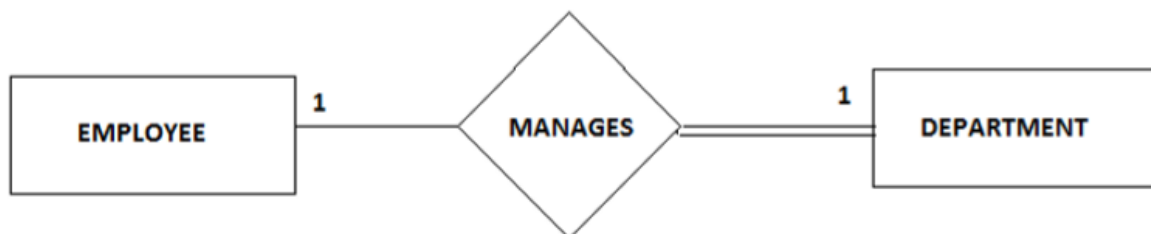


Consider two entities Employee and Department related via Works_For relationship. Now, every Employee works in at least one department therefore an Employee entity exist if it has at least one Works_For relationship with Department entity. Thus, the participation of Employee in Works_For is total relationship.

Total Participation is represented by double line in ER diagram.

Partial Participation – Each entity in entity set may or may not occur in at least one relationship in a relationship set.

For example: Consider two entities Employee and Department and they are related to each other via Manages relationship. An Employee must manage a Department, he or she could be the head of the department. But not every Employee in the company manages the department. So, participation of employee in the Manages relationship type is partial i.e. only a particular set of Employees will manage the Department but not all.



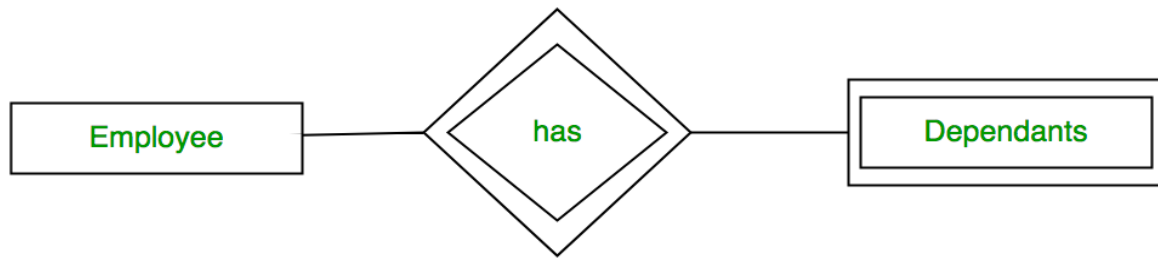
Weak Entity Type and Identifying Relationship:

An entity type has a key attribute which uniquely identifies each entity in the entity set. But there exists some entity type for which key attribute can't be defined. These are called Weak Entity type.

For example, A company may store the information of dependants (Parents, Children, Spouse) of an Employee. But the dependents don't have existence without the employee. So Dependent will be weak entity type and Employee will be Identifying Entity type for Dependant.

A weak entity type is represented by a double rectangle. The participation of weak entity type is always total. The relationship between weak entity type and

its identifying strong entity type is called identifying relationship and it is represented by double diamond.



Mapping Cardinality:

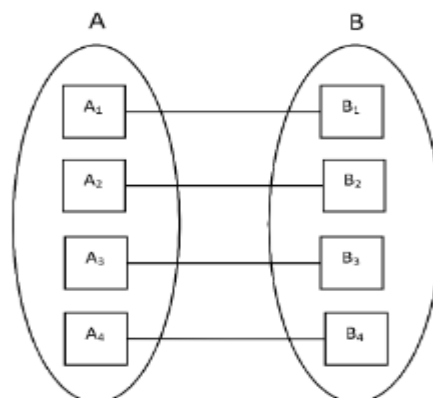
In a relational database model, the number of records in a table or relation is called cardinality in the E-R Model of the database. However, relationship types usually have certain constraints that limit the number of instance of one entity type that can be associated with each instance of another related entity type.

Therefore, the cardinality of a relationship is defined as; the number of instances of one entity type that can be associated with each instance of another related entity type.

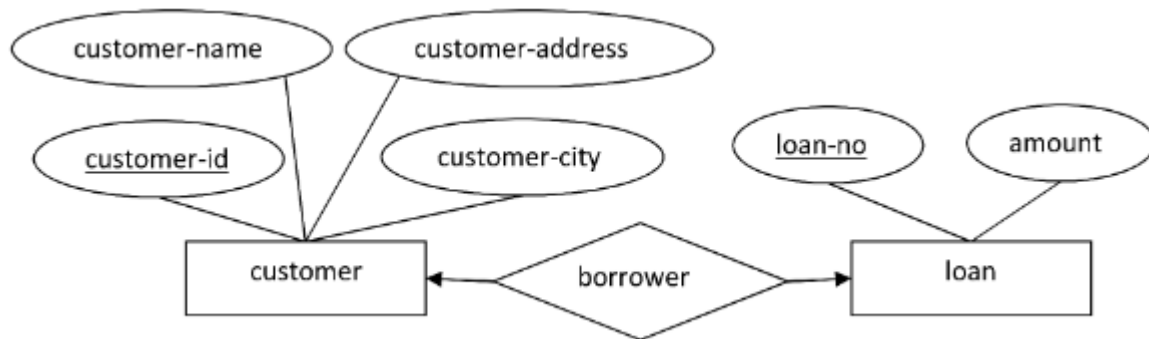
Suppose there are two entity types A and B that are connected by a relationship R in the E-R Model of the database of same data file. If there are no cardinality constraints on R, then any number of instance A can be associated with any number of instances of B.

The cardinality of one id denoted by a small vertical line (|) on the relationship line, next to the first entity type or before the second entity type.

One-to-one relationship

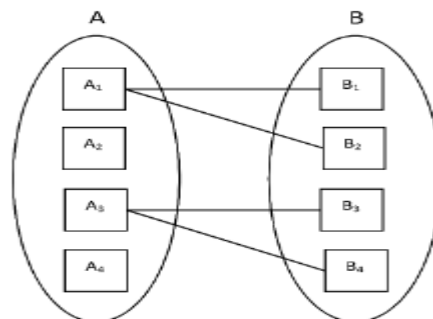


An entity in A is associated with at most (only) one entity in B and an entity in B is associated with at most (only) one entity in A.

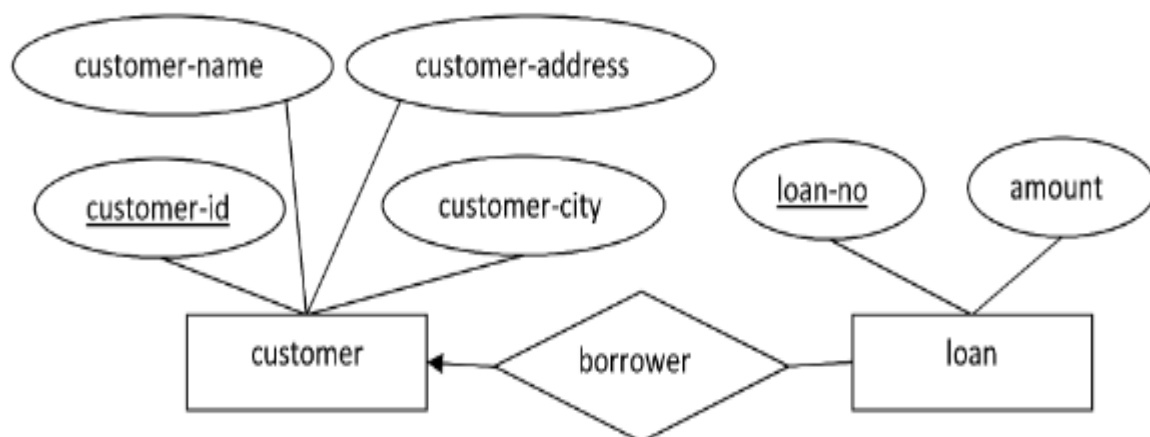


A customer is connected with only one loan using the relationship borrower and a loan is connected with only one customer using borrower.

One-to-many relationship

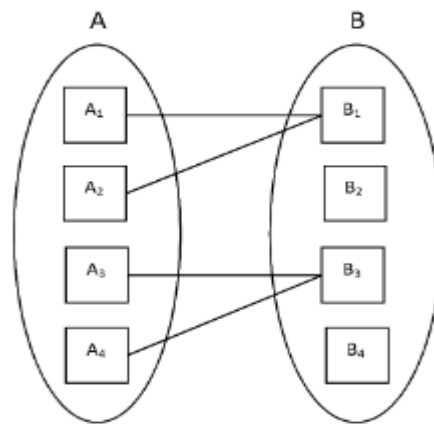


An entity in A is associated with any number (zero or more) of entities in B and an entity in B is associated with at most one (only) entity in A.

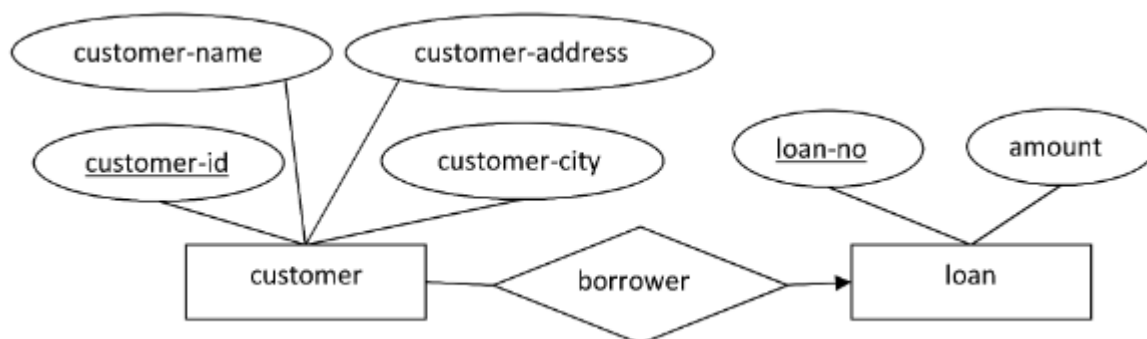


In this example, a loan is connected with only one customer using borrower and a customer is connected with more than one loans using borrower.

Many-to-one relationship

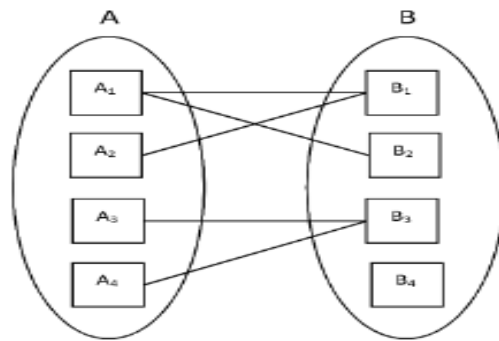


An entity in A is associated with at most (only) one entity in B and an entity in B is associated with any number (zero or more) of entities in A.

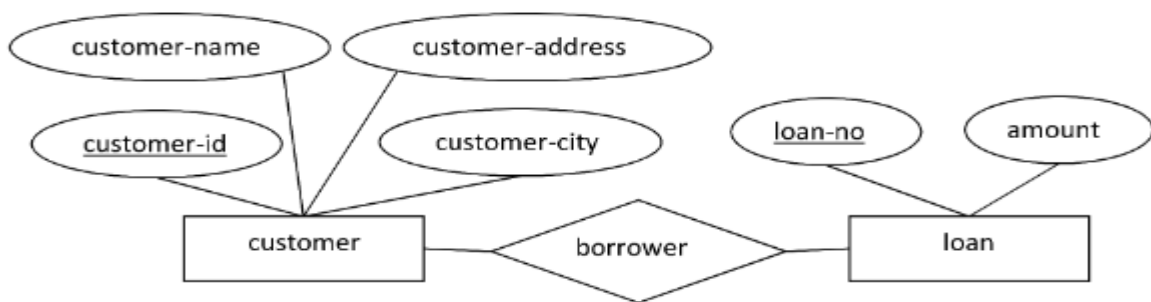


In a many-to-one relationship a loan is connected with more than one customer using borrower and a customer is connected with only one loan using borrower.

Many-to-many relationship



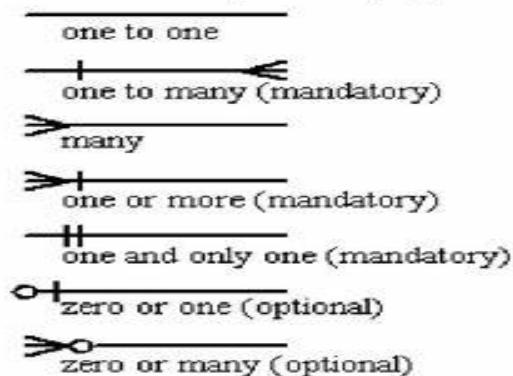
An entity in A is associated with any number (zero or more) of entities in B and an entity in B is associated with any number (zero or more) of entities in A.



A customer is connected with more than one loan using borrower and a loan is connected with more than one customer using borrower.

Cardinality Notations

Information Engineering style



Keys in E-R diagram:

KEYS in DBMS is an attribute or set of attributes which helps you to identify a row(tuple) in a relation(table). They allow you to find the relation between two tables. Keys help you uniquely identify a row in a table by a combination of one

or more columns in that table. Key is also helpful for finding unique record or row from the table. Database key is also helpful for finding unique record or row from the table.

Example:

Employee ID	FirstName	LastName
11	Andrew	Johnson
22	Tom	Wood
33	Alex	Hale

In the above-given example, employee ID is a primary key because it uniquely identifies an employee record. In this table, no other employee can have the same employee ID.

Why we need a Key?

Here are some reasons for using sql key in the DBMS system.

1. Keys help you to identify any row of data in a table. In a real-world application, a table could contain thousands of records. Moreover, the records could be duplicated. Keys ensure that you can uniquely identify a table record despite these challenges.
2. Allows you to establish a relationship between and identify the relation between tables
3. Help you to enforce identity and integrity in the relationship.

Types of Keys in Database Management System

There are mainly five different types of Keys in DBMS and each key has its different functionality:

- **Super Key** - A super key is a group of single or multiple keys which identifies rows in a table.
- **Primary Key** - is a column or group of columns in a table that uniquely identify every row in that table.
- **Candidate Key** - is a set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes.
- **Foreign Key** - is a column that creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity.
- **Composite Key** - when you have more than one column as part of primary key then that becomes composite key.

Primary Key: PRIMARY KEY is a column or group of columns in a table that uniquely identify every row in that table. The Primary Key can't be a duplicate meaning the same value can't appear more than once in the table. A table cannot have more than one primary key.

Rules for defining Primary key:

1. Two rows can't have the same primary key value.
2. It must for every row to have a primary key value.
3. The primary key field cannot be null.
4. The value in a primary key column can never be modified or updated if any foreign key refers to that primary key.

StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com

Example: In the following example, `StudID` is a Primary Key.

EmpSSN	EmpNum	Empname
9812345098	AB05	Shown

9876512345	AB06	Roslyn
199937890	AB07	James

Super key: A superkey is a group of single or multiple keys which identifies rows in a table. A Super key may have additional attributes that are not needed for unique identification.

Example:

In the above-given example, EmpSSN and EmpNum name are superkeys.

Candidate Key

CANDIDATE KEY is a set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes. The Primary key should be selected from the candidate keys. Every table must have at least a single candidate key. A table can have multiple candidate keys but only a single primary key.

Properties of Candidate key:

- It must contain unique values
- Candidate key may have multiple attributes
- Must not contain null values
- It should contain minimum fields to ensure uniqueness
- Uniquely identify each record in a table

Example: In the given table Stud ID, Roll No, and email are candidate keys

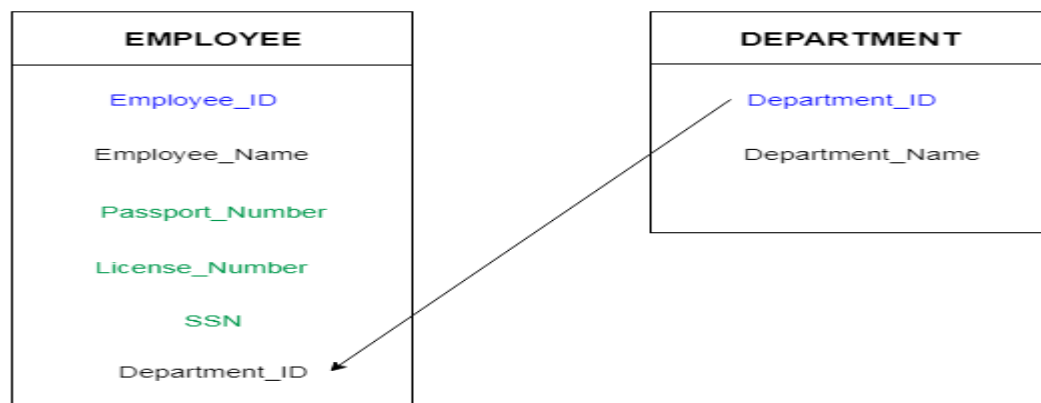
StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com

which help us to uniquely identify the student record in the table.

Foreign key

Foreign key is a column that creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity. It acts as a cross-reference between two tables as it references the primary key of another table.

In a company, every employee works in a specific department, and employee and department are two different entities. So we can't store the information of the department in the employee table. That's why we link these two tables through the primary key of one table. We add the primary key of the DEPARTMENT table, Department_Id as a new attribute in the EMPLOYEE table. Now in the EMPLOYEE table, Department_Id is the foreign key, and both the tables are related.



Composite key:

A primary key having two or more attributes is called composite key. It is a combination of two or more columns.

StudentID	StudentEnrollNo	StudentMarks	StudentPercentage
S001	0721722	570	90
S002	0721790	490	80
S003	0721766	440	86

Above, our composite keys are StudentID and StudentEnrollNo. The table has two attributes as primary key.

Therefore, the Primary Key consisting of two or more attribute is called Composite Key.

Importance of Entity-Relationship diagram

Entity-Relationship Diagram views real world as entities. It introduced in 1976 by P.P.Chen and is known as ER Diagram, E R Model, etc.

The following states the importance of Entity-Relationship diagram –

1. Assist the Database Developer: The ER Diagram assists the design even before the construction of tables begins.
2. Requirements Gathering: This helps the users to plan how to organize data.
3. Documentation Tool: ER Diagrams can work as a documentation to make others understand the core of the database.
4. Logic of the database: It communicates the logical structure of the database to the users.
5. Blueprint: An ER Diagram is considered as the blueprint of the database.

Extended E-R Features:

EER is a high-level data model that incorporates the extensions to the original ER model. Enhanced ERD are high level models that represent the requirements and complexities of complex database.

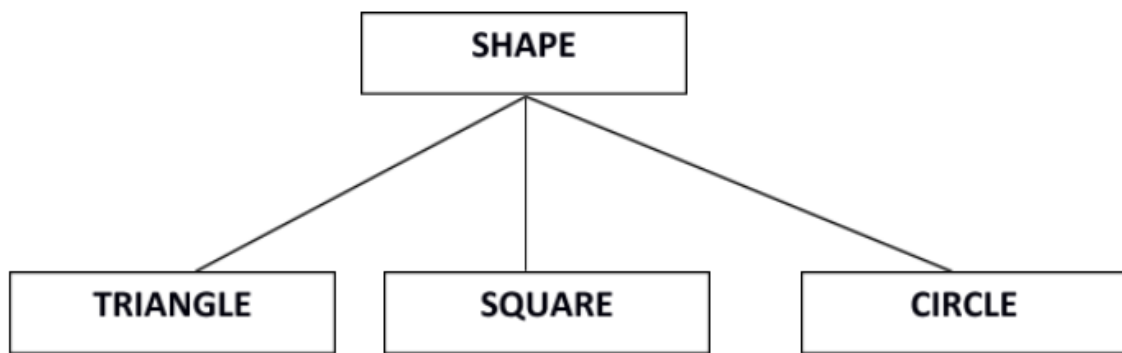
It is a diagrammatic technique for displaying the Sub Class and Super Class; Specialization and Generalization; Union or Category; Aggregation etc.

EE-R includes –

1. Subclasses and Super classes.
2. Specialization and Generalization.
3. Aggregation.

These concepts are used to create EE-R diagrams.

Subclasses and Super class: Super class is an entity that can be divided into further subtype. For **example** – consider Shape super class.



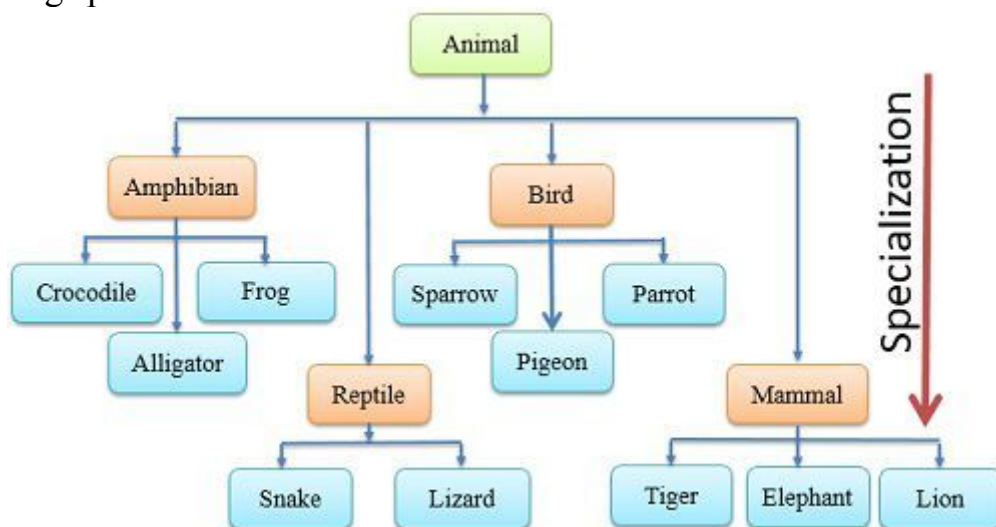
Super class shape has sub groups: Triangle, Square and Circle.

Sub classes are the group of entities with some unique attributes. Sub class inherits the properties and attributes from super class.

Specialisation: Specialization is a designing procedure that proceeds in a top-down manner. Specialization is just opposite to Generalization. In specialization, we split an entity to form multiple lower level entities. These newly formed lower level entities inherit some features of the higher level entities.

It may happen that a higher level entity may not split further and hence, it may not have any lower level entity. Specialization is always applied on a single entity, and if overruled, it increases the size of a schema.

Let us discuss specialization with the help of an example. Let us take an entity Animal and apply specialization on it. The entity animal can further be split into amphibian, reptiles, birds, mammals the list is long, but this much is enough for explaining specialization.



Now an entity amphibian can be further split to crocodile, alligator, frog. The entity reptile splits to snake, lizard. The entity bird can be split to sparrow, pigeon, parrot. Mammals can be split to a tiger, lion, elephant.

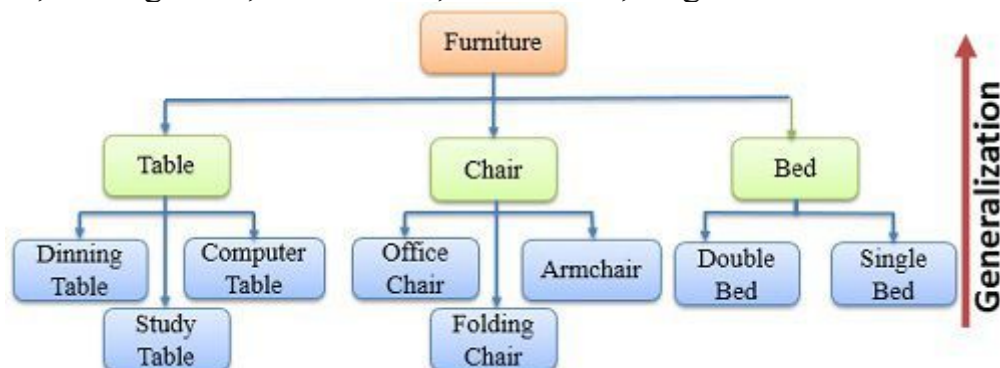
This is how specialization increases the number of entity increasing the size of a schema.

Generalization: Generalization, this term is often used while designing any relational Schema. If designing proceeds in a bottom-up manner then it is featured as Generalization. If the entities, that are figured out to create a schema shares some similar features, then they are combined to form a higher-level entity.

In generalization, we say if some lower level entities have some characteristics in common then they are clubbed to form a new higher level entity that will further combine with some entities to form a new higher level entity. In generalization, there can never be a high level entity without any lower level entity.

Generalization is always applied on a group of entities, and if overviewed it seems to reduce the size of a schema.

Let us discuss an example of generalization. If I ask you to name some furniture, then it's common to say study table, dining table, computer table, armchair, folding chair, office chair, double bed, single bed and the list is so on.



Now, we will generalise these furniture, here, study table, dining table, computer table, all are kind of table so, I will generalise these entities to a new higher level entity Table. The entities armchair, folding chair, office chair, are the kind of Chair so, they form a new higher level entity Chair. The entity Double bed, Single bed combined to form the higher level entity Bed. Now, we have three higher level entity Table, Chair and Bed which can be clubbed further to form a new higher level entity Furniture.

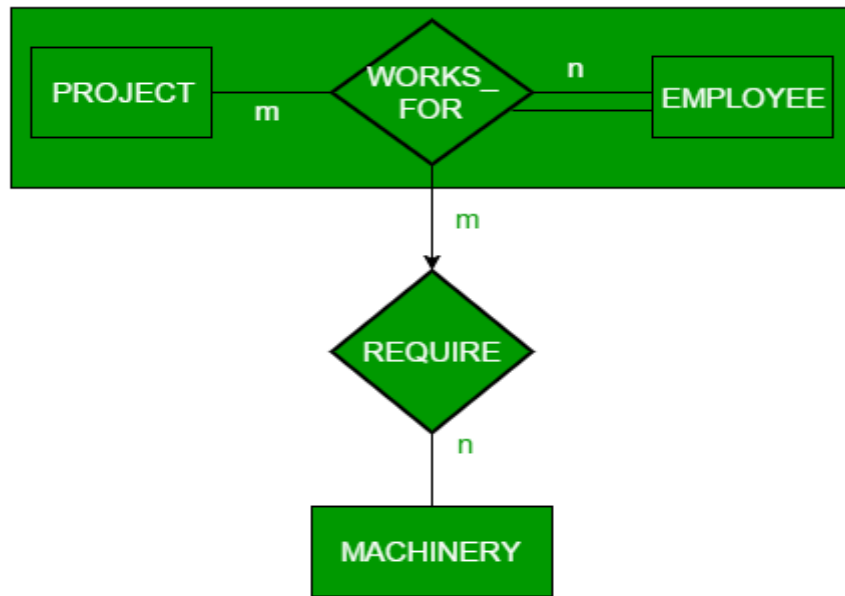
Furniture entity is the generalised entity of all those entities we discussed above.

Key Differences Between Generalization and Specialization in DBMS

1. The fundamental difference between generalization and specialization is that Generalization is a bottom-up approach. However, specialization is a top-down approach.
2. Generalization club all the entities that share some common properties to form a new entity. On the other hands, specialization spilt an entity to form multiple new entities that inherit some properties of the spilted entity.
3. In generalization, a higher entity must have some lower entities whereas, in specialization, a higher entity may not have any lower entity present.
4. Generalization helps in reducing the size of schema whereas, specialization is just opposite it increases the number of entities thereby increasing the size of a schema.
5. Generalization is always applied to the group of entities whereas, specialization is always applied on a single entity.
6. Generalization results in a formation of a single entity whereas, Specialization results in the formation of multiple new entities.

Aggregation: An ER diagram is not capable of representing relationship between an entity and a relationship which may be required in some scenarios. In those cases, a relationship with its corresponding entities is aggregated into a higher level entity.

For Example, Employee working for a project may require some machinery. So, REQUIRE relationship is needed between relationship WORKS_FOR and entity MACHINERY. Using aggregation, WORKS_FOR relationship with its entities EMPLOYEE and PROJECT is aggregated into single entity and relationship REQUIRE is created between aggregated entity and MACHINERY.



Aggregation

Reasons for Using Aggregation in DBMS:

Aggregation method in DBMS is applied when the DBMS has the below conditions,

- When the DBMS has only trivial entities and no complex entities, it is not possible for future upgrade or In this situation, two or more trivial entities can be used to create a new relationship between them, and the resulting artifact can be formed as a new complex entity.
- When the DBMS has one trivial entity, which needs to be used for more than one operation, then this entity is used to create a relationship with more than one other This can result in multiple aggregation processes, and multiple new entities depending on the operations applied between the entities.
- When the Entity-Relationship model is not able to represent the relationship between any entities in its system, that particular entity can be used to create a new relationship with any other The resulting entity can be used to set up relationships in the Entity-Relationship model. In this scenario, Aggregation helps in making use of all the entities in the system, as an alternative to letting any entity to be inoperative.

These are a few sample scenarios where Aggregation will be an essential process. Practically, there can be other possible situations where Aggregation can help in making the DBMS more scalable for future upgrades and long-lasting in nature.

E-R diagram design issues:

Use of Entity Sets versus Attributes

Consider the entity set employee with attributes employee-name and telephone-number. It can easily be argued that a telephone is an entity in its own right with attributes telephone-number and location (the office where the telephone is located). If we take this point of view, we must redefine the employee entity set as:

- The employee entity set with attribute employee-name
- The telephone entity set with attributes telephone-number and location
- The relationship set emp-telephone, which denotes the association between employees and the telephones that they have

What, then, is the main difference between these two definitions of an employee? Treating a telephone as an attribute telephone-number implies that employees have precisely one telephone number each. Treating a telephone as an entity telephone permits employees to have several telephone numbers (including zero) associated with them. However, we could instead easily define telephone-number as a multivalued attribute to allow multiple telephones per employee.

The main difference then is that treating a telephone as an entity better models a situation where one may want to keep extra information about a telephone, such as its location, or its type (mobile, video phone, or plain old telephone), or who all share the telephone. Thus, treating telephone as an entity is more general than treating it as an attribute and is appropriate when the generality may be useful.

In contrast, it would not be appropriate to treat the attribute employee-name as an entity; it is difficult to argue that employee-name is an entity in its own right (in contrast to the telephone). Thus, it is appropriate to have employee-name as an attribute of the employee entity set.

Two natural questions thus arise: What constitutes an attribute, and what constitutes an entity set? Unfortunately, there are no simple answers. The distinctions mainly depend on the structure of the real-world enterprise being modeled, and on the semantics associated with the attribute in question.

A common mistake is to use the primary key of an entity set as an attribute of another entity set, instead of using a relationship. For example, it is incorrect to model customer-id as an attribute of loan even if each loan had only one customer. The relationship borrower is the correct way to represent the connection between loans and customers, since it makes their connection explicit, rather than implicit via an attribute.

Another related mistake that people sometimes make is to designate the primary key attributes of the related entity sets as attributes of the relationship set. This

should not be done, since the primary key attributes are already implicit in the relationship.

Use of Entity Sets versus Relationship Sets

It is not always clear whether an object is best expressed by an entity set or a relationship set. We assumed that a bank loan is modeled as an entity. An alternative is to model a loan not as an entity, but rather as a relationship between customers and branches, with *loan-number* and *amount* as descriptive attributes. Each loan is represented by a relationship between a customer and a branch.

If every loan is held by exactly one customer and is associated with exactly one branch, we may find satisfactory the design where a loan is represented as a relationship. However, with this design, we cannot represent conveniently a situation in which several customers hold a loan jointly. To handle such a situation, we must define a separate relationship for each holder of the joint loan. Then, we must replicate the values for the descriptive attributes *loan-number* and *amount* in each such relationship. Each such relationship must, of course, have the same value for the descriptive attributes *loan-number* and *amount*.

Two problems arise as a result of the replication: (1) the data are stored multiple times, wasting storage space, and (2) updates potentially leave the data in an inconsistent state, where the values differ in two relationships for attributes that are supposed to have the same value.

One possible guideline in determining whether to use an entity set or a relationship set is to designate a relationship set to describe an action that occurs between entities. This approach can also be useful in deciding whether certain attributes may be more appropriately expressed as relationships.

Binary versus nary Relationship Sets

Relationships in databases are often binary. Some relationships that appear to be nonbinary could actually be better represented by several binary relationships. For instance, one could create a ternary relationship parent, relating a child to his/her mother and father. However, such a relationship could also be represented by two binary relationships, mother and father, relating a child to his/her mother and father separately. Using the two relationships mother and father allows us record a child's mother, even if we are not aware of the father's identity; a null value would be required if the ternary relationship parent is used. Using binary relationship sets is preferable in this case.

In fact, it is always possible to replace a nonbinary (n -ary, for $n > 2$) relationship set by a number of distinct binary relationship sets. For simplicity, consider the abstract ternary ($n = 3$) relationship set R , relating entity sets A , B ,

and C. We replace the relationship set R by an entity set E, and create three relationship sets:

- RA, relating E and A
- RB, relating E and B
- RC, relating E and C

If the relationship set R had any attributes, these are assigned to entity set E; further, a special identifying attribute is created for E (since it must be possible to distinguish different entities in an entity set on the basis of their attribute values). For each relationship (ai, bi, ci) in the relationship set R, we create a new entity ei in the entity set E. Then, in each of the three new relationship sets, we insert a relationship as follows:

- (ei, ai) in RA
- (ei, bi) in RB
- (ei, ci) in RC

We can generalize this process in a straightforward manner to n-ary relationship sets. Thus, conceptually, we can restrict the E-R model to include only binary relationship sets. However, this restriction is not always desirable.

- An identifying attribute may have to be created for the entity set created to represent the relationship set. This attribute, along with the extra relationship sets required, increases the complexity of the design and overall storage requirements.
- A n-ary relationship set shows more clearly that several entities participate in a single relationship.
- There may not be a way to translate constraints on the ternary relationship into constraints on the binary relationships. For example, consider a constraint that says that R is many-to-one from A, B to C; that is, each pair of entities from A and B is associated with at most one C entity. This constraint cannot be expressed by using cardinality constraints on the relationship sets RA, RB, and RC.

Consider the relationship set works-on, relating employee, branch, and job. We cannot directly split works-on into binary relationships between employee and branch and between employee and job. If we did so, we would be able to record that Jones is a manager and an auditor and that Jones works at Perryridge and Down-town; however, we would not be able to record that Jones is a manager at Perryridge and an auditor at Downtown, but is not an auditor at Perryridge or a manager at Downtown.

The relationship set works-on can be split into binary relationships by creating a new entity set as described above. However, doing so would not be very natural.

Placement of Relationship Attributes

The cardinality ratio of a relationship can affect the placement of relationship attributes. Thus, attributes of one-to-one or one-to-many relationship sets can be associated with one of the participating entity sets, rather than with the

relationship set. For instance, let us specify that depositor is a one-to-many relationship set such that one customer may have several accounts, but each account is held by only one customer. In this case, the attribute access-date, which specifies when the customer last accessed that account, could be associated with the account entity set, only some of the attributes of the two entity sets are shown. Since each account entity participates in a relationship with at most one instance of customer, making this attribute designation would have the same meaning

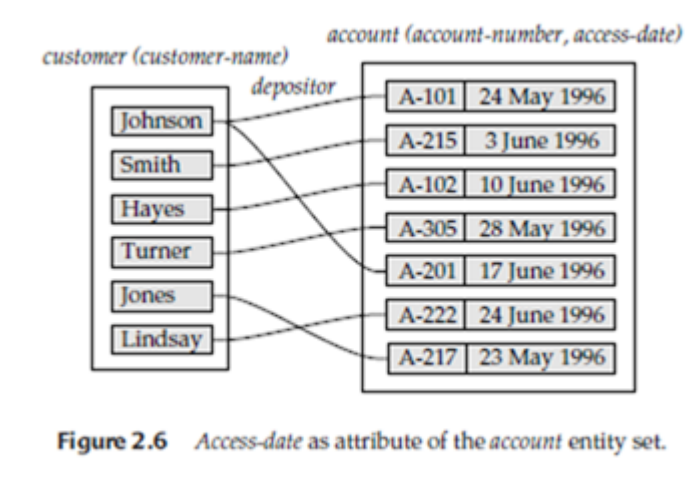


Figure 2.6 Access-date as attribute of the account entity set.

as would placing access-date with the depositor relationship set. Attributes of a one-to-many relationship set can be repositioned to only the entity set on the “many” side of the relationship. For one-to-one relationship sets, on the other hand, the relationship attribute can be associated with either one of the participating entities.

The design decision of where to place descriptive attributes in such cases — as a relationship or entity attribute — should reflect the characteristics of the enterprise being modeled. The designer may choose to retain access-date as an attribute of depositor to express explicitly that an access occurs at the point of interaction between the customer and account entity sets.

The choice of attribute placement is more clear-cut for many-to-many relationship sets. Returning to our example, let us specify the perhaps more realistic case that depositor is a many-to-many relationship set expressing that a customer may have one or more accounts, and that an account can be held by one or more customers.

If we are to express the date on which a specific customer last accessed a specific account, access-date must be an attribute of the depositor relationship set, rather than either one of the participating entities. If access-date were an attribute of account, for instance, we could not determine which customer made the most recent access to a joint account. When an attribute is determined by the combination of participating entity sets, rather than by either entity separately, that attribute must be associated with the many-to-many relationship set.

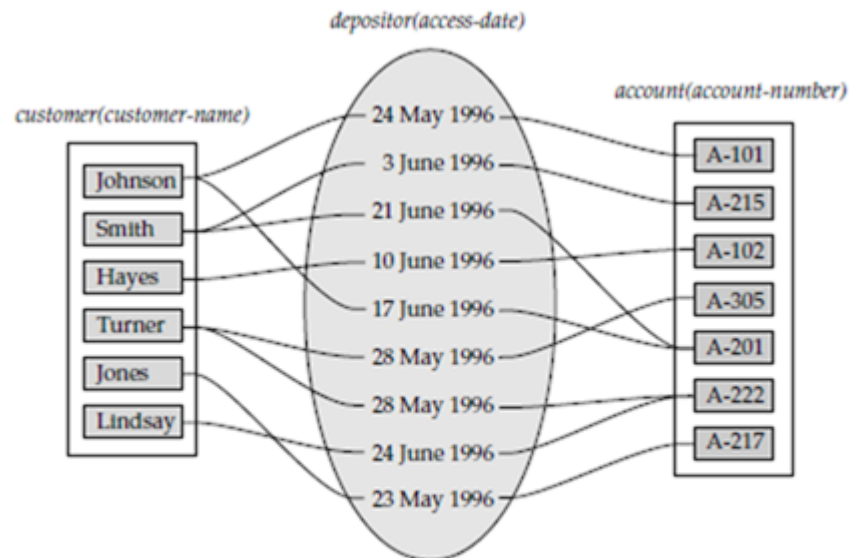


Figure 2.7 Access-date as attribute of the depositor relationship set.

ER Diagram to Relational Model Conversion:

First step of any relational database design is to make ER Diagram for it and then convert it into relational Model. Relational Model represents how data is stored in database in the form of table.



Lets learn step by step how to convert ER diagram into relational model

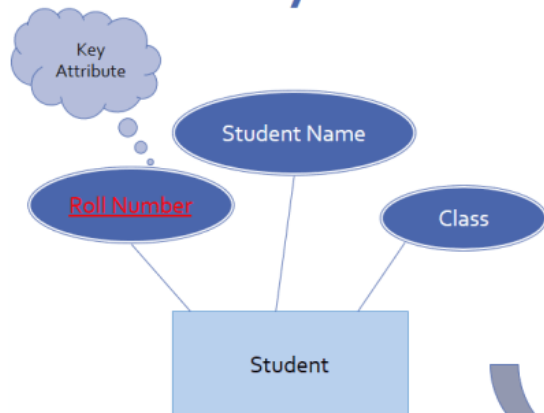
1.Entity Set:

Consider we have entity STUDENT in ER diagram with attributes Roll Number, Student Name and Class.

To convert this entity set into relational schema

- 1.Entity is mapped as relation in Relational schema
- 2.Attributes of Entity set are mapped as attributes for that Relation.
- 3.Key attribute of Entity becomes Primary key for that Relation.

1.Entity Set



Relational Schema

Student(Roll Number, Student Name, Class)

Primary Key

2.Entity set with multi valued attribute:

Consider we have entity set Employee with attributes Employee ID, Name and Contact number.

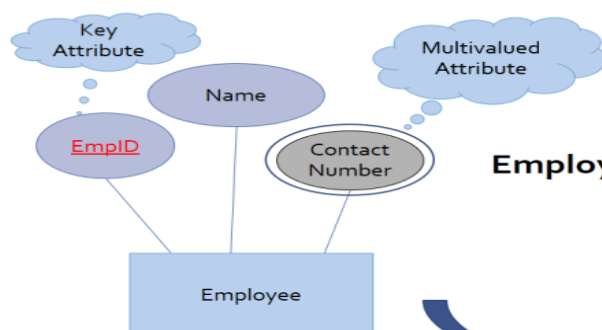
Here contact number is multivalued attribute as it has multiple values. as an employee can have more than one contact number for that we have to repeat all attributes for every new contact number. This will lead to data redundancy in table.

Hence to convert entity with multivalued attribute into relational schema separate relation is created for multivalued attribute in which

1.Key attribute and multivalued attribute of entity set becomes primary key of relation.

2.Separate relation employee is created with remaining attributes.

Due to this instead of repeating all attributes of entity now only one attribute is need to repeat.



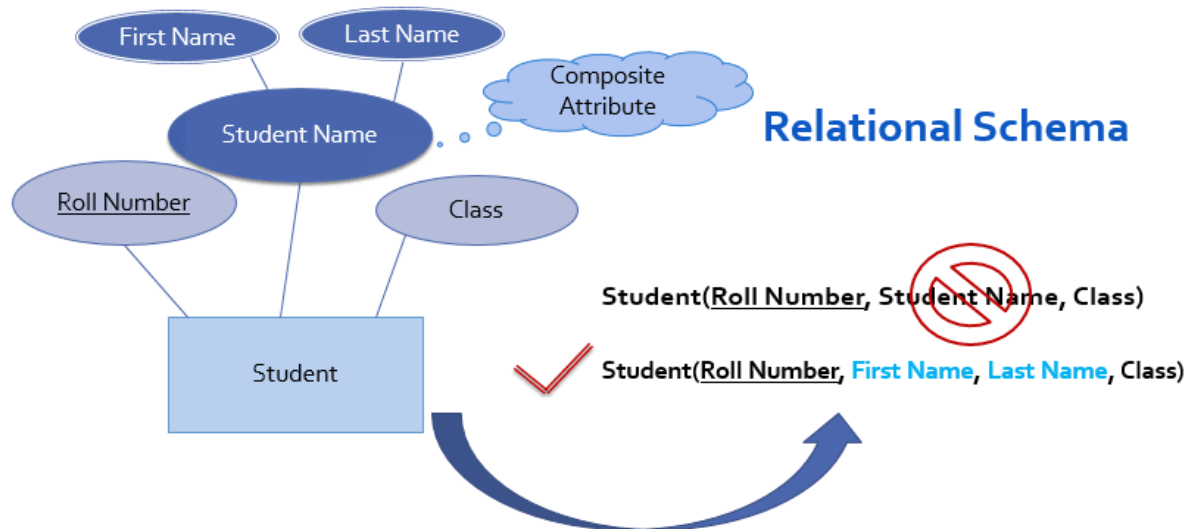
Relational Schema

EmployeeContact(EmpID, Contact Number)

3.Entity set with Composite attribute:

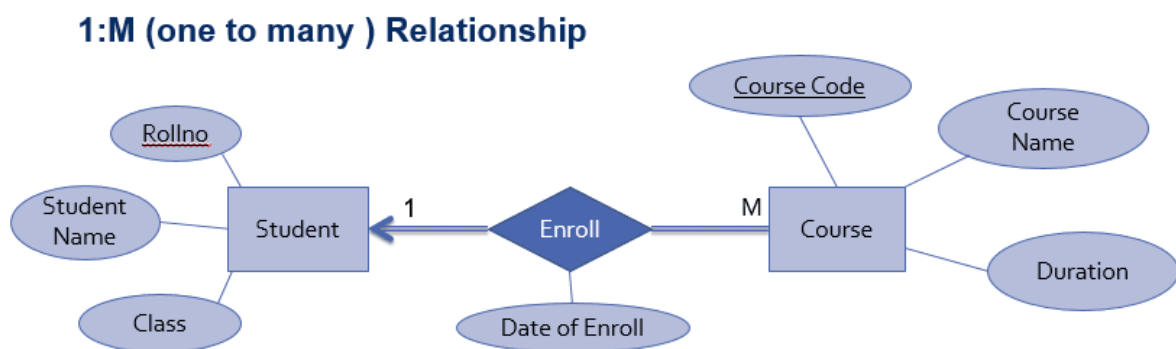
Consider entity set student with attributes Roll Number, Student Name and Class. here student name is composite attribute as it has further divided into First name, last name.

In this case to convert entity into relational schema, composite attribute student name should not be include in relation but all parts of composite attribute are mapped as simple attributes for relation.



4. 1:M (one to many) Relationship:

Consider 1:M relationship set enrolled exist between entity sets student and course as follow,



Attributes of entity set student are Roll no which is primary key, student name and class

Attributes of entity set course are Course code which is primary key, Course name and duration

And date of enroll is attribute of relationship set enroll.

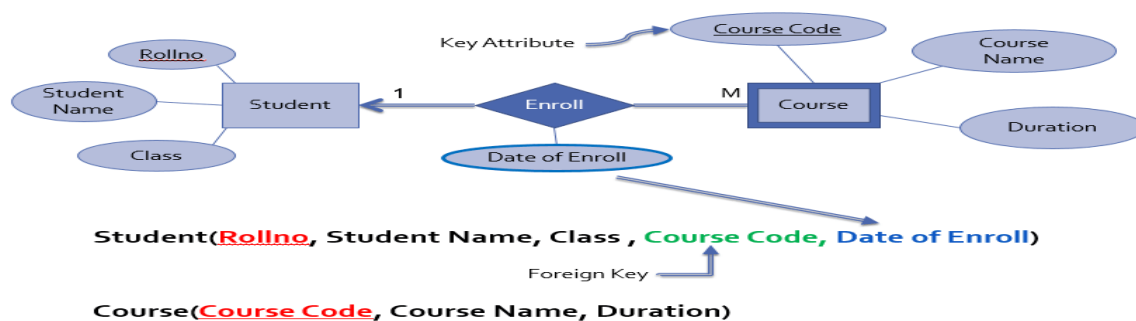
Here Enroll is 1:M relationship exist between entity set student and course which means that one student can enroll in multiple courses

In this case to convert this relationship into relational schema,

1. Separate relation is created for all participating entity sets (student and course)

2. Key attribute of Many's side entity set (course) is mapped as foreign key in one's side relation (Student)

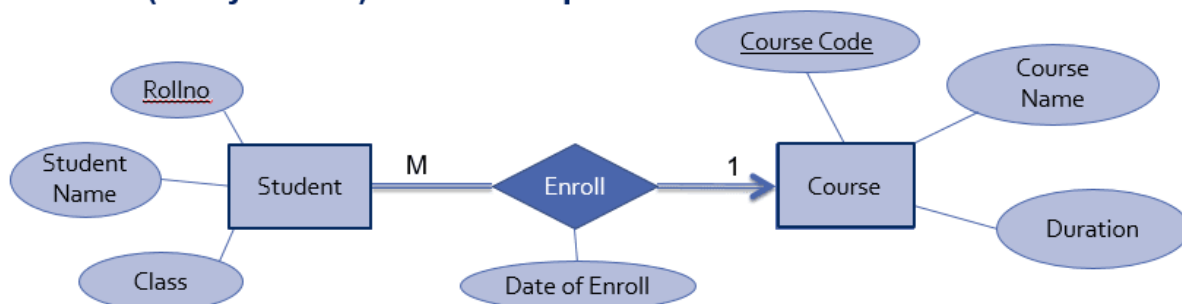
3. All attributes of relationship set are mapped as attributes for relation of one's side entity set (student)



5. M:1 (many to one) Relationship:

Consider same relationship set enroll exist between entity sets student and course . but here student is many side entity set while course is one side entity set. Which means many student can enroll in one course.

M:1 (many to one) Relationship

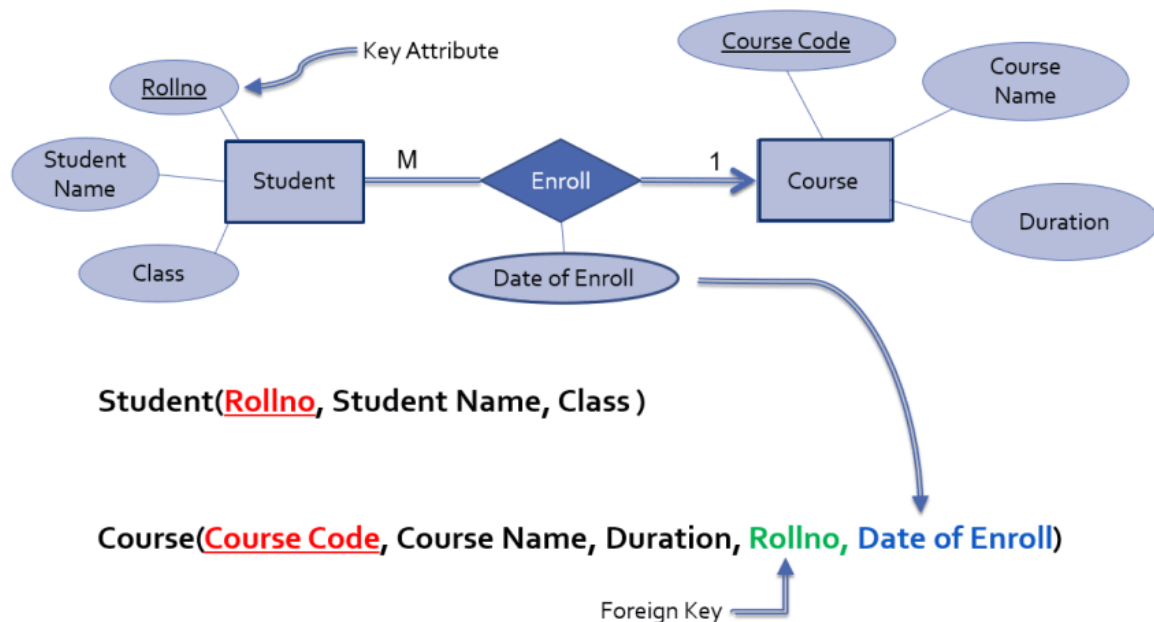


To convert this relationship set into relational schema,

1. Separate relation is created for all participating entity sets.

2. Key attribute of Many's side entity set student is mapped as foreign key in one's side relation

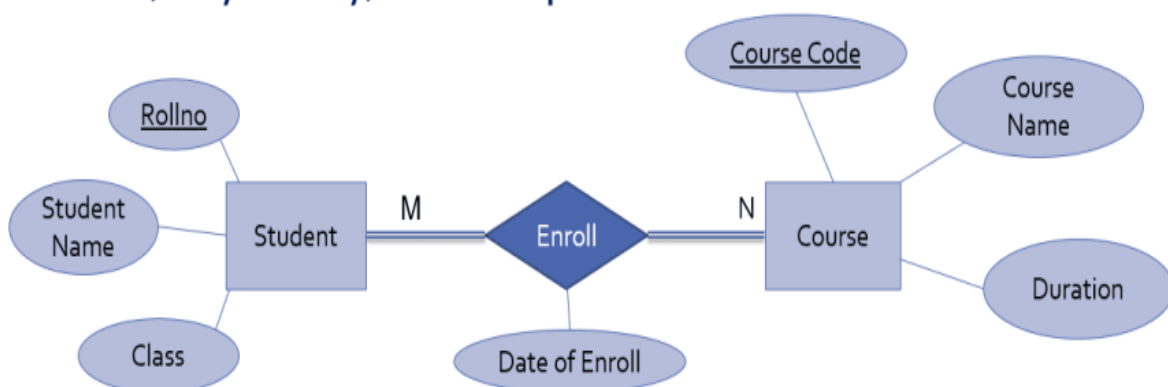
3. All attributes of relationship set are mapped as attributes for one's side relation course.



6. M:N (many to many) Relationship:

Consider same relationship set enrolled exist between entity sets student and course, which means multiple student can enroll in multiple courses.

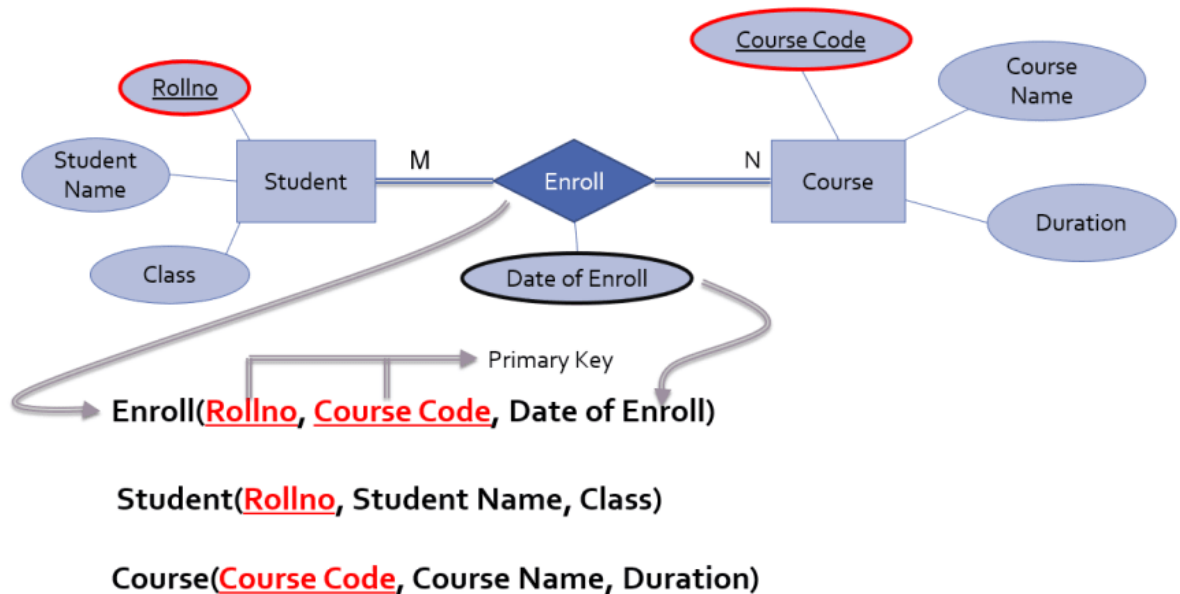
M:N (many to many) Relationship



To convert this Relationship set into relational schema,

1. Relationship set is mapped as separate relation

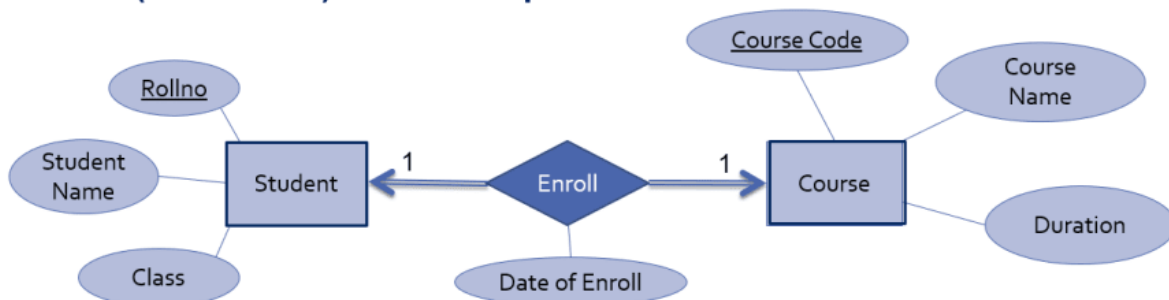
2. Key attributes of participating entity sets are mapped as primary key for that relation
3. Attribute of relationship set becomes simple attributes for that relation
4. And separate relation is created for other participating entities



7. 1:1 (one to one) Relationship:

Consider same relationship set enroll exist between entity sets student and course ,which means one student can enroll in only one courses

1:1 (One to one) Relationship



To convert this Relationship set into relational schema,

1. Separate relation is created for all participating entity sets.
2. Primary Key of Relation Student can be act as foreign key for relation Course OR Primary Key of Relation Course act as foreign key for relation Student.

Student(Rollno, Student Name, Class)

Course(Course Code, Course Name, Duration, Rollno)

Foreign Key

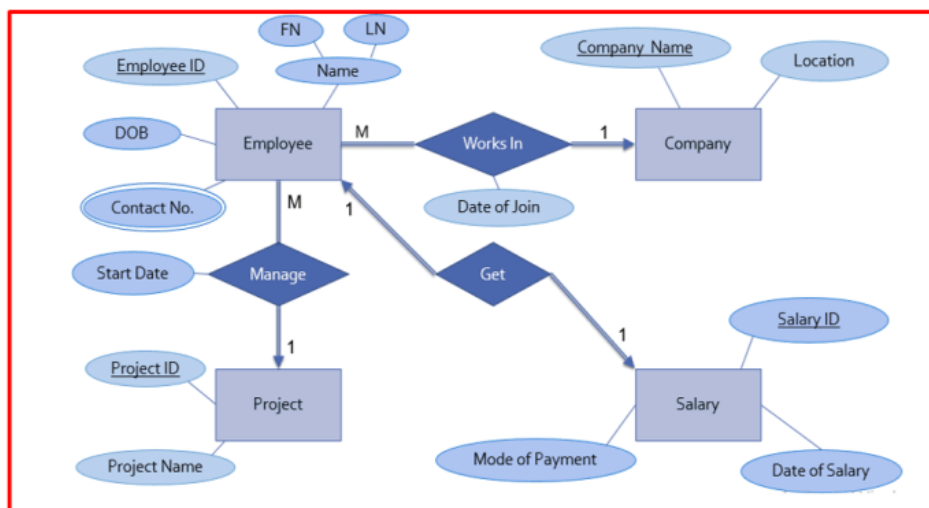
OR

Student(Rollno, Student Name, Class, Course Code)

Foreign Key

Course(Course Code, Course Name, Duration)

Example:



ER Diagram

Employee(Employee ID , DOB , FN , LN , Salary ID)
 EmployeeContact(Employee ID , Contact No)
 Company(Company Name , Location , Employee ID , Date of Join)
 Project(Project ID , Project Name , Employee ID , Start Date)
 Salary(Salary ID , Mode of Payment , Date of Salary , Employee ID)

Relational Model

